Protokoły sieciowe i projekty backendowe

Plan nauki dla backend developera w networkingu · Go, Rust, C++

Opracowano 6 listopada 2026 · wymagania zebrane z ogłoszeń Cloudflare, Ubiquiti, Akamai, Cisco, Motorola i firm telekomunikacyjnych

1. Jak korzystać z tego planu

- **Zasada:** pisz protokół według RFC, a potem sprawdzaj go prawdziwymi narzędziami (*dig, curl, tcpdump, Wireshark, FRR, SIPp, radtest*). Dzięki temu możesz powiedzieć, że Twój serwer odpowiada na *dig*, a nie że „zna DNS".
- **Kolejność języków:** zacznij od **Go** (szybkie efekty, wymagany w ogłoszeniach), **Rusta** dołącz po 3–4 tygodniach na protokole, który już znasz (np. DNS przeniesiony z Go). **C++** traktuj jako trzeci język, jeśli celujesz w Motorolę, Ericssona lub Cisco z C++.
- **Jakość zamiast ilości:** każdy protokół ma mieć testy na prawdziwych pakietach, fuzzing parsera, pomiar wydajności i krótki README. Kryptografii nie pisz od zera. TLS, QUIC i SSH bierz z bibliotek.
- **Czas jest orientacyjny.** Plan zakłada kilka godzin dziennie. Kolejność możesz zmieniać zależnie od oferty, na którą celujesz.

Legenda języków: Go · Rust · C++ · Python (prototypy, testy, skrypty) · Node/TS (Node.js i TypeScript). W tabeli pierwszy język jest rekomendowany, kolejne to sensowne alternatywy.

2. Protokoły do zaprogramowania od zera

Protokół (RFC)	Języki	Co zbudować	Sprawdź czym	Przydaje się w
POZIOM 1 · Podstawy (tygodnie 1–3)				
1. TFTP **RFC 1350**	Go · Rust · C++ · Python	Klient i serwer po UDP: RRQ/WRQ, bloki po 512 B, ACK, retransmisje i timeouty. Dobra rozgrzewka.	tftp-hpa, Wireshark	UDP, automat stanów, timeouty
2. TCP i HTTP/1.1 **RFC 9293, 9112**	Go (bez net/http) · Rust · C++	Serwer TCP z ramkowaniem i timeoutami. Potem własny serwer HTTP: parser żądań, nagłówki, keep-alive, chunked, kilka tras.	curl, wrk, k6, Wireshark	Proxy, API, każdy backend
3. WebSocket **RFC 6455**	Go · Rust · Node/TS	Handshake Upgrade, ramki, maskowanie, ping/pong, zamykanie połączenia.	websocat, przeglądarka	Dashboardy, UniFi, powiadomienia
4. DNS **RFC 1034, 1035, 6891 (EDNS)**	Go, potem port do Rust · C++	Parser i builder komunikatów z kompresją nazw, stub resolver, serwer autorytatywny po UDP i TCP, cache z TTL.	dig, kdig, Wireshark	Cloudflare, ISP, projekt 3
5. DHCP **RFC 2131, 2132**	Go · Rust · C++	Serwer z pulą adresów, dzierżawami i opcjami, API REST do zarządzania (IPAM).	dhclient w namespace sieciowym, Wireshark	ISP, sieci, IPAM
POZIOM 2 · Pakiety i warstwy niższe (tygodnie 4–6)				
6. Ethernet, ARP, IPv4, ICMP, UDP, TCP **RFC 826, 791, 792, 768, 9293**	Rust · Go (gopacket) · C++ (libpcap)	Parser surowych bajtów z plików pcap, własny ping i traceroute, wykrywanie ARP spoofingu.	Wireshark, tcpdump	Pakiety, eBPF, IDS, bezpieczeństwo
7. NTP **RFC 5905**	Go · Rust · C++	Klient SNTP i prosty serwer: offset, opóźnienie, tryby, stratum.	chronyc, ntpdate -q	Systemy, telemetria czasu
8. STUN **RFC 8489**	Go · Rust · Node/TS	Klient i serwer: XOR-MAPPED-ADDRESS, wykrywanie typu NAT.	Wireshark, publiczny serwer STUN	WebRTC, VoIP
9. TLS 1.3 (tylko ClientHello) **RFC 8446**	Rust · Go	Parser handshake: SNI, ALPN, wersje, odcisk JA3. Bez pisania kryptografii.	openssl s_client, curl -v	Bezpieczeństwo, filtrowanie ruchu
10. Niezawodny transport po UDP (inspiracja: RFC 9293, 9000)	Rust · Go · C++	Okno przesuwne, retransmisje, kontrola przeciążenia, symulowana utrata pakietów.	tc netem, iperf3	TCP i QUIC od środka, warstwa L2
11. GRE i VXLAN **RFC 2784, 7348**	Rust · Go · C++	Enkapsulacja przez urządzenie TUN, dwa hosty w osobnych namespace'ach.	ip, tcpdump	SD-WAN, VPN, chmura
POZIOM 3 · Routing i telemetria (tygodnie 6–9)				
12. BGP-4 **RFC 4271, BMP: RFC 7854**	Go · Rust · C++	Automat stanów, OPEN, KEEPALIVE, UPDATE, NOTIFICATION, tablica tras. Sesja z FRR w containerlab.	FRR (vtysh), Wireshark	Cloudflare, operatorzy, projekt 2

Protokół (RFC)	Języki	Co zbudować	Sprawdź czym	Przydaje się w
13. OSPFv2 (zaawansowany) RFC 2328	Go · Rust	Hello, sąsiedztwa, LSA typu 1, algorytm SPF.	FRR, Wireshark	Routing, sieci operatorskie
14. SNMP v2c RFC 3416	Go · Python · Rust	Agent z kilkoma OID i menedżer: GET, GETNEXT, WALK, kodowanie BER.	snmpwalk, snmpget	Monitoring, ISP
15. sFlow v5 / IPFIX RFC 7011	Go · Rust	Kolektor datagramów, agregacja przepływów, zapis do ClickHouse.	hsflowd, softflowd	Telemetria, projekt 2
16. Syslog RFC 5424	Go · Rust	Odbiornik UDP i TCP, parser, indeks, proste zapytania.	logger, rsyslog	Obserwowalność, Splunk
POZIOM 4 · ISP i telekomunikacja (tygodnie 9–10 lub wg celu)				
17. RADIUS RFC 2865, 2866 (accounting)	Go · Rust · C++ · Java	Serwer Access-Request / Accept / Reject, atrybuty, Message-Authenticator, accounting. Abonenci w PostgreSQL, rozliczanie ruchu do ClickHouse.	radtest, eapol_test, FreeRADIUS jako klient	ISP, WiFi, UniFi
18. SIP + SDP RFC 3261, 8866	Go · Rust · Node/TS · C++ (PJSIP)	Parser, REGISTER z uwierzytelnianiem digest, INVITE / 200 / ACK / BYE, lokalizacja użytkowników (Redis), routing połączeń.	SIPp, Linphone, Wireshark	VoIP, Ubiquiti Talk, Cisco, Motorola
19. RTP / RTCP RFC 3550	Go · Rust · C++	Odbiornik, bufor jitter, statystyki strat i jitteru, prosty przekaźnik RTP.	SIPp (media), ffmpeg, Wireshark	Media, telekomy, WebRTC
20. MQTT 3.1.1 / 5.0 specyfikacja OASIS	Go · Rust · Node/TS	Broker: CONNECT, PUBLISH, SUBSCRIBE, QoS 0 i 1, retain.	mosquitto_pub / mosquitto_sub	IoT, UniFi Connect

POZIOM 5 · Używaj gotowych bibliotek (nie pisz od zera)

Co	Biblioteki	Dlaczego nie od zera
QUIC i HTTP/3	quic-go (Go), quinn (Rust)	Ogromna specyfikacja, a wiedza o transporcie z punktu 10 wystarczy
TLS	crypto/tls (Go), rustls (Rust)	Kryptografii nie pisze się samodzielnie. To ryzyko bezpieczeństwa
SSH i NETCONF	x/crypto/ssh (Go), russh (Rust)	Kryptografia. Skup się na kliencie i automatyzacji konfiguracji
gRPC i gNMI	grpc-go, tonic (Rust)	Wystarczy umieć zaprojektować API w protobuf
WireGuard / Noise	wireguard-go, boringtun	Tunel z punktu 11 zbuduj, ale szyfrowanie bierz z biblioteki

3. Projekty główne (osobne repozytoria)

Trzy projekty łączą protokoły z poprzedniej sekcji w systemy, o które pytają ogłoszenia. Każdy ma etapy (MVP, wersja 1, wersja 2), więc możesz go pokazać już po pierwszym etapie.

Projekt 1 · edge-proxy: reverse proxy z płaszczyzną sterowania

Cel	Zbudować proxy L7, którym steruje się przez API, tak jak robią to bramki API i CDN.
Komponenty	**Data plane:** proxy HTTP/1.1 i HTTP/2 z terminacją TLS, load balancing (round-robin, least-connections), health checki, limit żądań (token bucket), circuit breaker, timeouty. **Control plane:** REST i gRPC do zarządzania trasami, upstreamami, certyfikatami i politykami. **Dane:** konfiguracja w PostgreSQL, liczniki limitów w Redis.
Technologie	Prometheus i Grafana, OAuth2 / JWT na API, Docker, Kubernetes (kind), Helm, GitHub Actions, test obciążeniowy k6.
Języki	**Go** (główny, całość). **Rust** (tokio, hyper, axum): alternatywa lub data plane w wersji 2. **C++** (Boost.Beast, Asio): możliwe, ale więcej pracy.
Etapy	**MVP (tydz. 1–2):** proxy HTTP/1.1 z round-robin i health checkami, konfiguracja z pliku. **v1 (tydz. 5):** control plane REST, PostgreSQL, przeładowanie konfiguracji bez zrywania połączeń, limit żądań w Redis, metryki. **v2:** TLS, HTTP/2, JWT, Helm i Kubernetes, test k6, runbook awarii.
Kryteria gotowości (do README)	Zmierzona przepustowość i opóźnienie p99 (liczby w README). Przeładowanie konfiguracji nie przerywa żądań. Po wyłączeniu upstreamu ruch przechodzi na zdrowe instancje. Pokrycie testami i fuzzing parsera.
Pokrywa wymagania	Cloudflare (proxy TLS/HTTP, API, Go), Akamai (brama API, rate limiting, load balancing, Kubernetes), Ubiquiti (OAuth, Docker, SQL, defensywne programowanie).

Projekt 2 · netops-platform: telemetria i konfiguracja sieci

Cel	Zbierać dane z urządzeń sieciowych, przechowywać i wersjonować ich konfigurację oraz alarmować o awariach.
Komponenty	**Laboratorium:** containerlab z 3–4 routerami FRR (BGP, OSPF). **Kolektory:** SNMP, sFlow, konfiguracja przez SSH i NETCONF, monitor BGP (sesja z punktu 12 lub BMP). **Przetwarzanie:** kolejka NATS lub Kafka, agregator. **Dane:** ClickHouse (przepływy), PostgreSQL (inwentarz, historia konfiguracji z diffem i przywracaniem). **API** REST i gRPC, reguły alertów.
Technologie	Grafana, Prometheus, Docker Compose lub Kubernetes, GitHub Actions, testy integracyjne na laboratorium.
Języki	**Go** (główny). **Rust** (kolektory sFlow i SNMP). **Python** (skrypty laboratorium, testy integracyjne). **C++** możliwe dla kolektorów.
Etapy	**MVP (tydz. 7):** inwentarz urządzeń, pobieranie konfiguracji przez SSH, zapis wersji i diff. **v1:** SNMP, metryki, dashboard Grafana. **v2 (tydz. 8–9):** sFlow → NATS → ClickHouse, monitor BGP, alerty.
Kryteria gotowości (do README)	Wyłączenie łącza w laboratorium wywołuje alert w mniej niż 30 s. Historia zmian konfiguracji z diffem. Wykres przepływów z ClickHouse. Skrypt stawiający całe laboratorium jednym poleceniem.
Pokrywa wymagania	Cloudflare (zespoły Traffic i Network Platforms: ClickHouse, NETCONF, Grafana, PostgreSQL, BGP), Cisco, Arista, Juniper (automatyzacja i telemetria sieci).

Projekt 3 · dns-xdp: serwer DNS z ochroną w jądrze

Cel	Zbudować serwer DNS i filtr pakietów w jądrze Linuksa, który ogranicza ruch zanim trafi do aplikacji.
Komponenty	**Serwer:** autorytatywny z plików stref plus cache, UDP i TCP, EDNS, limit rozmiaru odpowiedzi. **Filtr:** program XDP/eBPF ograniczający pakiety na adres źródłowy i liczący statystyki w mapach BPF. **Pomiary:** generator ruchu (dnsperf), wykres pakietów na sekundę, metryki w Grafanie.
Technologie	eBPF/XDP, mapy BPF, tokio, Prometheus, Docker, fuzzing (cargo-fuzz), CI.
Języki	**Rust** (główny: tokio plus aya). **Go** (cilium/ebpf). **C** / **C++** z libbpf: najbliżej jądra, jeśli celujesz w firmy z C/C++.
Etapy	**MVP (tydz. 4–5):** serwer z plików stref, port parsera z Go. **v1:** cache, TCP, EDNS, metryki. **v2 (tydz. 8–9):** filtr XDP, testy przepustowości z filtrem i bez niego.
Kryteria gotowości (do README)	Wykres pakietów na sekundę z XDP i bez. Parser DNS odporny na fuzzing. Serwer odpowiada na dig dla stref testowych. Opis, co się dzieje przy ataku zalewem zapytań.
Pokrywa wymagania	Cloudflare (DNS, eBPF/XDP, Rust, programowanie systemowe), firmy sieciowe z C/C++ (Motorola, Ericsson).

4. Projekty opcjonalne (pod konkretny cel)

Opcja A · isp-provisioning: serwis abonentów dla ISP

Cel	Zasymulować to, co robi backend operatora: logowanie abonentów, przydział adresów i rozliczanie.
Komponenty	Serwer RADIUS (RFC 2865, 2866) z abonentami w PostgreSQL i planami prędkości przekazywanymi jako atrybuty RADIUS. Serwer DHCP z punktu 5. Rozliczanie sesji (accounting) do ClickHouse, raport zużycia, REST do panelu. Opcjonalnie Change of Authorization (RFC 5176).
Technologie	PostgreSQL, ClickHouse, Docker, Grafana, FreeRADIUS jako klient testowy.
Języki	**Go** (główny). **Rust**, **C++**. **Java** bywa częsta w systemach operatorskich (OSS/BSS), ale to wiedza ogólna, nie z ogłoszeń, które sprawdzałem.
Etapy	**MVP:** Access-Request z bazy abonentów. **v1:** accounting i raporty. **v2:** plany i CoA.
Kryteria gotowości (do README)	radtest i eapol_test przechodzą. Raport zużycia dla abonenta. Test obciążeniowy liczby uwierzytelnień na sekundę.
Pokrywa wymagania	Firmy ISP i WiFi, Ubiquiti (UniFi).

Opcja B · voip-gateway: rejestrator SIP z przekaźnikiem RTP

Cel	Zbudować mały system VoIP: rejestracja, nawiązywanie połączeń i przekazywanie mediów.
Komponenty	Rejestrator i proxy SIP (punkt 18) z uwierzytelnianiem digest, lokalizacją użytkowników w Redisie i regułami routingu. Przekaźnik RTP (punkt 19) ze statystykami jitteru i strat. Opcjonalnie bramka WebRTC. REST do zarządzania i eksport rekordów połączeń (CDR).
Technologie	Redis, PostgreSQL, SIPp do testów, Wireshark, Docker.
Języki	**Go** (biblioteka Pion do WebRTC). **Node/TS** (Ubiquiti UniFi Talk używa Node.js). **Rust**. **C++** (PJSIP).
Etapy	**MVP:** REGISTER i INVITE między dwoma klientami. **v1:** przekaźnik RTP i statystyki. **v2:** WebRTC i CDR.
Kryteria gotowości (do README)	Rozmowa dwóch klientów Linphone przez Twój serwer. Scenariusz SIPp w CI. Wykres jitteru i strat.
Pokrywa wymagania	Ubiquiti (UniFi Talk), Cisco (Webex), Motorola (telekomunikacja).

5. Które wymaganie z ogłoszeń ćwiczysz gdzie

Wymaganie z ogłoszeń	Gdzie to ćwiczysz	Firmy
Go i/lub Rust	Wszystkie projekty. Go: 1 i 2, Rust: 3	Cloudflare, Ubiquiti
TCP/UDP, DNS, TLS, HTTP, QUIC	Protokoły 2, 4, 9 i 10, projekt 1 i 3	Cloudflare
eBPF / XDP, przetwarzanie pakietów	Protokół 6, projekt 3	Cloudflare
BGP, routing, NETCONF, sFlow	Protokoły 12, 13, 15, projekt 2	Cloudflare, Cisco, Arista
REST i gRPC API, systemy rozproszone	Projekty 1 i 2	Wszystkie
PostgreSQL, ClickHouse, Redis	Projekty 1, 2 i opcja A	Cloudflare, Ubiquiti
Docker, Kubernetes, CI/CD	Projekty 1 i 2	Akamai, Cloudflare, Ubiquiti
Prometheus, Grafana, obserwowalność	Wszystkie projekty	Cloudflare, Akamai, Cisco
OAuth, OWASP, bezpieczeństwo API	Projekt 1 (JWT, OAuth2)	Ubiquiti, Cisco
SIP, RTP, WebRTC	Protokoły 18 i 19, opcja B	Ubiquiti, Cisco, Motorola
Dyżury on-call, odpowiedzialność za działanie	Runbook i alerty w projektach 1 i 2	Cloudflare, Cisco

6. Plan na 10 tygodni

Tydzień	Co robisz	Efekt do pokazania
1	Konfiguracja Go, TFTP, TCP, początek serwera HTTP	Repozytorium protocols-from-scratch z pierwszymi testami
2	HTTP/1.1 dokończony, MVP edge-proxy	Proxy z round-robin i health checkami
3	DNS w Go, DHCP	Serwer DNS odpowiadający na dig
4	Nauka Rusta: parser pakietów (6), port DNS do Rust	Parser pcap w Rust, początek projektu 3
5	edge-proxy v1: control plane, PostgreSQL, Redis	API do zarządzania proxy i metryki
6	BGP w Go, laboratorium containerlab z FRR	Sesja BGP z FRR widoczna w vtysh
7	netops-platform MVP: inwentarz, SSH, diff konfiguracji	Historia konfiguracji urządzeń
8	sFlow do ClickHouse, filtr XDP w projekcie 3	Wykres przepływów, pierwsze pomiary XDP
9	Alerty, Grafana, testy przepustowości XDP	Wykresy z liczbami w README
10	RADIUS, SIP, RTP (wg celu), dopracowanie README, CI, profil GitHub	Kompletne repozytoria i profil z przypiętymi projektami

7. Lista kontrolna dla każdego repozytorium

- **README:** opis (co i po co), diagram architektury, uruchomienie jednym poleceniem, wyniki testów wydajności z liczbami, sekcja „ograniczenia”.
- **CI w GitHub Actions:** testy, lint, build i zielony badge.
- **Testy:** jednostkowe, na prawdziwych przechwyconych pakietach i integracyjne. Dla parserów też fuzzing (*go test -fuzz*, *cargo-fuzz*).
- **Struktura repozytoriów:** trzy osobne (edge-proxy, netops-platform, dns-xdp) oraz jedno wspólne *protocols-from-scratch* z katalogami go/ i rust/, w którym każdy protokół ma własne README.
- **Historia commitów:** małe, opisowe zmiany. Licencja (np. MIT) i tematy (topics) na GitHubie.
- **Uczciwość:** mów, że umiesz technologię, dopiero gdy projekt działa i jest na GitHubie. Na rozmowie pytają o szczegóły: dlaczego tak, jak mierzyłeś, co poszło źle. Jeśli używasz narzędzi AI do kodowania, rozumiej każdy fragment.

Uwaga: wymagania w tabeli pochodzą z ogłoszeń, które w większości są już zamknięte (Cloudflare, Ubiquiti, Akamai). Traktuj je jako wzorzec oczekiwań, a nie aktualną ofertę. Część dotycząca ISP opiera się na ogólnej wiedzy, nie na przeanalizowanych ogłoszeniach.